

Cuaderno de Actividades de Aula: Conectores JDBC y Oracle

Guía de Prácticas Guiadas Síncronas (Teoría en Pizarra → Código en Vivo en IDE)

MÓDULO	UNIDAD	CICLO / CURSO	PROFESOR
Acceso a Datos (0486)	UT2: JDBC y Oracle	2º DAM (2026-2027)	Sergio Capdevila Díez

Bloque 1 · Conectividad y CRUD

Actividad 2.1: Conexión Inicial y Diagnóstico con Oracle Database 23c

BÁSICA · 25 MINUTOS

Alineación Curricular: RA2 (CE2.a, CE2.b) **Entorno:** Java 21 · Oracle 23c · Maven

Configuración de las dependencias Maven para el driver oficial Oracle JDBC (ojdbc11), creación de una clase de utilidad de conexión con DriverManager, y verificación de los metadatos de versión y compatibilidad del motor.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>DriverManager.getConnection(url, user, pass)</code>	Abre un socket TCP contra el listener de Oracle en el puerto 1521, autentica al usuario c##dam2_user y establece la sesión JDBC.
<code>Connection.getMetaData().getDatabaseProductVersion()</code>	Consulta el catálogo del motor y devuelve la cadena con la versión exacta de Oracle Database (ej. Oracle Database 23c Free Release).
<code>Connection.close()</code> (try-with-resources)	Cierra la sesión y libera los recursos en el servidor de base de datos.

ENUNCIADO DEL RETO DE AULA

Crea un proyecto Maven llamado `ad-ut2-conectores` e incorpora la dependencia `com.oracle.database.jdbc:ojdbc11:23.3.0.23.09`. Desarrolla una clase `ConexionOracleTest` que solicite los parámetros de conexión y confirme el estado de la base de datos imprimiendo la versión del motor y el catálogo predeterminado.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionOracleTest {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        System.out.println("=== TEST CONECTIVIDAD ORACLE 23c ===");
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            DatabaseMetaData meta = conn.getMetaData();
            System.out.println("[OK] Conexión establecida exitosamente.");
            System.out.println("Motor: " + meta.getDatabaseProductName());
            System.out.println("Versión: " + meta.getDatabaseProductVersion());
            System.out.println("Driver JDBC: " + meta.getDriverName() + " v" + meta.getDriverVersion());
            System.out.println("AutoCommit por defecto: " + conn.getAutoCommit());
        } catch (SQLException e) {
```

```
        System.err.println("[ERROR] No se pudo conectar a Oracle: " + e.getMessage());
        System.err.println("Código Error Oracle: " + e.getErrorCode());
        System.err.println("SQLState: " + e.getSQLState());
    }
}
```

Actividad 2.2: Operaciones CRUD con PreparedStatement y Claves

INTERMEDIA · 35 MINUTOS

Autogeneradas

Alineación Curricular: RA2 (CE2.c, CE2.d) **Entorno:** Java 21 · Oracle 23c · Maven

Creación de la tabla de productos con columna identity en Oracle, inserción de registros parametrizados recuperando el ID generado con `Statement.RETURN_GENERATED_KEYS`, actualización de stock y eliminación controlada.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)</code>	Precompila la sentencia SQL de inserción habilitando el canal de retorno para los identificadores generados automáticamente.
<code>ps.setBigDecimal(index, valor) /</code> <code>ps.setString(index, valor)</code>	Asigna parámetros fuertemente tipados a las posiciones '?' de la sentencia precompilada sin riesgo de inyección SQL.
<code>ps.executeUpdate()</code>	Ejecuta la sentencia DML en Oracle y retorna el número de filas afectadas en la base de datos.
<code>ps.getGeneratedKeys()</code>	Retorna un ResultSet especial que contiene las claves primarias autogeneradas por la base de datos.

ENUNCIADO DEL RETO DE AULA

Crea la tabla `productos` en tu esquema Oracle con columnas `id\_producto` NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY, `nombre` VARCHAR2(100), `precio` NUMBER(10,2), `stock` NUMBER(5). Desarrolla una clase `GestionProductosCRUD` que inserte 2 productos recuperando sus IDs generados, actualice el precio del primero y verifique el conteo de filas afectadas.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import java.math.BigDecimal;
import java.sql.*;

public class GestionProductosCRUD {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        String sqlInsert = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";
        String sqlUpdate = "UPDATE productos SET precio = ? WHERE id_producto = ?";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            // 1. Inserción con recuperación de ID
            long idGenerado = -1;
            try (PreparedStatement ps = conn.prepareStatement(sqlInsert, Statement.RETURN_GENERATED_KEYS)) {
```

```

        ps.setString(1, "Monitor Curvo 27 Samsung");
        ps.setBigDecimal(2, new BigDecimal("249.99"));
        ps.setInt(3, 8);

        int filas = ps.executeUpdate();
        System.out.println("Filas insertadas: " + filas);

        try (ResultSet rsKeys = ps.getGeneratedKeys()) {
            if (rsKeys.next()) {
                idGenerado = rsKeys.getLong(1);
                System.out.println("ID autogenerado por Oracle: " + idGenerado);
            }
        }
    }

    // 2. Actualización de precio
    if (idGenerado > 0) {
        try (PreparedStatement psUp = conn.prepareStatement(sqlUpdate)) {
            psUp.setBigDecimal(1, new BigDecimal("229.50"));
            psUp.setLong(2, idGenerado);
            int act = psUp.executeUpdate();
            System.out.println("Filas actualizadas para ID " + idGenerado + ": " + act);
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

Actividad 2.3: Navegación de ResultSet y Tratamiento Estricto de NULLs

INTERMEDIA · 30 MINUTOS

Alineación Curricular: RA2 (CE2.c, CE2.e) **Entorno:** Java 21 · Oracle 23c · Maven

Consulta con filtros dinámicos, navegación mediante ResultSet, obtención tipada de fechas con la API moderna java.time (LocalDate) y resolución segura de columnas numéricas que admiten valores NULL en Oracle.

🔧 INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>ResultSet.next()</code>	Avanza el cursor interno a la siguiente fila de resultados. Retorna true si hay registro, false al finalizar.
<code>ResultSet.isNull()</code>	Comprueba si la última columna primitiva leída (getInt, getDouble, getLong) contenía un valor NULL en la base de datos.
<code>ResultSet.getObject(col, Class)</code>	Método moderno de JDBC 4.2 que deserializa directamente tipos de datos complejos y permite retornar null para wrappers como Integer, Double o LocalDate.

🎯 ENUNCIADO DEL RETO DE AULA

Añade una columna `descuento NUMBER(5,2)` a la tabla `productos` que admita nulos. Inserta un producto sin descuento y otro con 10% de descuento. Desarrolla un programa que consulte todos los productos e imprima en consola si el descuento es NULL o tiene un valor numérico real, utilizando tanto `isNull()` como `rs.getObject("descuento", Double.class)`.

🔑 SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import java.math.BigDecimal;
import java.sql.*;

public class ConsultaProductosNullSafe {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        String sql = "SELECT id_producto, nombre, precio, descuento FROM productos ORDER BY id_producto";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
            PreparedStatement ps = conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()) {

            System.out.println("ID | NOMBRE | PRECIO | DESCUENTO");
            System.out.println("----+-----+-----+-----");
            while (rs.next()) {
                long id = rs.getLong("id_producto");
```

```
String nombre = rs.getString("nombre");
BigDecimal precio = rs.getBigDecimal("precio");

// Lectura moderna con getObject (permite null natural)
Double descuento = rs.getObject("descuento", Double.class);
String descStr = (descuento != null) ? String.format("%.2f%%", descuento) : "[SIN DESCUENTO]";

System.out.printf("%-4d| %-30s| %-9.2f| %s\n", id, nombre, precio, descStr);
    }
} catch (SQLException e) {
    e.printStackTrace();
}
}
```



Actividad 2.4: Invocación de Procedimientos Almacenados PL/SQL

AVANZADA · 35 MINUTOS

Alineación Curricular: RA2 (CE2.f) **Entorno:** Java 21 · Oracle 23c · Maven

Creación de un procedimiento PL/SQL en Oracle con parámetros IN y OUT, e invocación desde Java mediante CallableStatement, registrando los tipos SQL de salida con java.sql.Types.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>conn.prepareCall("{ call sp_nombre(?, ?) }")</code>	Crea la sentencia invocadora de subprogramas en el servidor de base de datos.
<code>cs.setLong(index, valor)</code>	Asigna valores a los parámetros formales de entrada (IN).
<code>cs.registerOutParameter(index, Types.TIPO)</code>	Declara ante el driver JDBC qué tipo de dato SQL se espera recibir de un parámetro de salida (OUT).
<code>cs.execute()</code>	Ejecuta el bloque PL/SQL en el motor Oracle.
<code>cs.getBigDecimal(index) /</code> <code>cs.getString(index)</code>	Extrae el valor devuelto por el parámetro de salida tras la ejecución.

ENUNCIADO DEL RETO DE AULA

Crea en Oracle el procedimiento ``sp_resumen_producto(p_id IN NUMBER, p_precio OUT NUMBER, p_estado OUT VARCHAR2)``. Si el stock es `> 0` el estado es 'DISPONIBLE', si es `0` 'AGOTADO'. Desarrolla una clase Java ``InvocacionPLSQLTest`` que ejecute el procedimiento para un producto dado y muestre sus parámetros OUT.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import java.math.BigDecimal;
import java.sql.*;

public class InvocacionPLSQLTest {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        // En Oracle:
        // CREATE OR REPLACE PROCEDURE sp_resumen_producto(
        //   p_id IN NUMBER, p_precio OUT NUMBER, p_estado OUT VARCHAR2
        // ) AS
        //   v_stock NUMBER;
        // BEGIN
        //   SELECT precio, stock INTO p_precio, v_stock FROM productos WHERE id_producto = p_id;
        //   IF v_stock > 0 THEN p_estado := 'DISPONIBLE'; ELSE p_estado := 'AGOTADO'; END IF;
        // END;
```

```

String callSql = "{ call sp_resumen_producto(?, ?, ?) }";
long idProductoABuscar = 1L;

try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
    CallableStatement cs = conn.prepareCall(callSql)) {

    cs.setLong(1, idProductoABuscar);           // IN
    cs.registerOutParameter(2, Types.NUMERIC); // OUT precio
    cs.registerOutParameter(3, Types.VARCHAR);  // OUT estado

    cs.execute();

    BigDecimal precio = cs.getBigDecimal(2);
    String estado = cs.getString(3);

    System.out.println("Resultado Procedimiento para ID " + idProductoABuscar + ":");
    System.out.println("Precio devuelto: " + precio + " €");
    System.out.println("Estado de stock: " + estado);

} catch (SQLException e) {
    e.printStackTrace();
}
}

```

Bloque 2 • Transacciones y Pools

Actividad 2.5: Gestión Transaccional ACID Bancaria y Savepoints

AVANZADA · 40 MINUTOS

Alineación Curricular: RA2 (CE2.d, CE2.g) **Entorno:** Java 21 · Oracle 23c · Maven

Control manual de transacciones para garantizar atomicidad absoluta en transferencias de fondos entre cuentas bancarias. Uso de `setAutoCommit(false)`, `commit()`, `rollback()` en bloque `catch` y creación de `Savepoints` intermedios.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>conn.setAutoCommit(false)</code>	Desactiva la confirmación inmediata automática. Comienza un bloque transaccional atómico.
<code>conn.setSavepoint("nombre")</code>	Establece un punto de restauración intermedio dentro de la transacción activa.
<code>conn.rollback(savepoint)</code>	Deshace exclusivamente las operaciones realizadas con posterioridad al savepoint indicado.
<code>conn.commit()</code>	Confirma de manera permanente y atómica todas las operaciones de la transacción en el almacenamiento de Oracle.
<code>conn.setAutoCommit(true)</code>	Restaura el estado original de la conexión en el bloque <code>finally</code> .

ENUNCIADO DEL RETO DE AULA

Crea una tabla `cuentas_bancarias` (`id` NUMBER PRIMARY KEY, `titular` VARCHAR2(50), `saldo` NUMBER(10,2)). Inserta dos cuentas con 1.000€ cada una. Realiza una transferencia transaccional de 300€ de la cuenta 1 a la cuenta 2. Simula un fallo de negocio (saldo insuficiente o excepción) y comprueba que ningún saldo queda modificado gracias al `rollback`.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import java.math.BigDecimal;
import java.sql.*;

public class TransferenciaBancariaTransaccional {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        realizarTransferencia(1L, 2L, new BigDecimal("300.00"), false);
        // Descomentar para simular fallo y verificar rollback:
        // realizarTransferencia(1L, 2L, new BigDecimal("2000.00"), true);
    }

    public static void realizarTransferencia(long origen, long destino, BigDecimal importe, boolean simularFallo) {
        String sqlDebitar = "UPDATE cuentas_bancarias SET saldo = saldo - ? WHERE id = ? AND saldo >= ?";
        String sqlAcreditar = "UPDATE cuentas_bancarias SET saldo = saldo + ? WHERE id = ?";

        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            conn.setAutoCommit(false);
            try {
                conn.setSavepoint("antes");
                PreparedStatement stmtDebitar = conn.prepareStatement(sqlDebitar);
                stmtDebitar.setLong(1, origen);
                stmtDebitar.setLong(2, origen);
                stmtDebitar.setBigDecimal(3, importe);
                stmtDebitar.executeUpdate();

                PreparedStatement stmtAcreditar = conn.prepareStatement(sqlAcreditar);
                stmtAcreditar.setLong(1, destino);
                stmtAcreditar.setLong(2, destino);
                stmtAcreditar.setBigDecimal(3, importe);
                stmtAcreditar.executeUpdate();

                if (simularFallo) {
                    throw new RuntimeException("Simulando fallo de negocio: saldo insuficiente");
                }

                conn.commit();
            } catch (SQLException e) {
                conn.rollback("antes");
                e.printStackTrace();
            }
        }
    }
}
```

```

conn.setAutoCommit(false); // 1. Iniciar transacción manual
try {
    // Paso A: Retirar fondos de cuenta origen
    try (PreparedStatement psDeb = conn.prepareStatement(sqlDebitar)) {
        psDeb.setBigDecimal(1, importe);
        psDeb.setLong(2, origen);
        psDeb.setBigDecimal(3, importe); // Restricción saldo suficiente
        int filas = psDeb.executeUpdate();
        if (filas == 0) {
            throw new IllegalStateException("Saldo insuficiente o cuenta origen inexistente.");
        }
    }

    if (simularFallo) {
        throw new RuntimeException("¡Fallo catastrófico simulado antes de acreditar!");
    }

    // Paso B: Ingresar fondos en cuenta destino
    try (PreparedStatement psAcr = conn.prepareStatement(sqlAcreditar)) {
        psAcr.setBigDecimal(1, importe);
        psAcr.setLong(2, destino);
        int filas = psAcr.executeUpdate();
        if (filas == 0) {
            throw new IllegalStateException("Cuenta destino no encontrada.");
        }
    }

    conn.commit(); // 2. Confirmación atómica
    System.out.println("[OK] Transferencia de " + importe + " € completada exitosamente.");
} catch (Exception e) {
    System.err.println("[ROLLBACK] Error en la transferencia: " + e.getMessage());
    conn.rollback(); // 3. Deshacer cualquier cambio intermedio
} finally {
    conn.setAutoCommit(true); // 4. Restaurar estado
}
} catch (SQLException e) {
    e.printStackTrace();
}
}

```

Actividad 2.6: Introspección Dinámica con DatabaseMetaData y ResultSetMetaData

INTERMEDIA · 30 MINUTOS

Alineación Curricular: RA2 (CE2.b, CE2.c) **Entorno:** Java 21 · Oracle 23c · Maven

Extracción dinámica de las tablas del esquema mediante DatabaseMetaData y visualización de columnas, tipos SQL y dimensiones de cualquier consulta genérica con ResultSetMetaData.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>conn.getMetaData().getTables(cat, schema, tablePattern, types)</code>	Consulta el catálogo del motor y retorna las tablas que coinciden con los filtros.
<code>ResultSet.getMetaData()</code>	Retorna la interfaz ResultSetMetaData asociada a una consulta activa.
<code>ResultSetMetaData.getColumnCount()</code>	Devuelve el número total de columnas proyectadas en el SELECT.
<code>ResultSetMetaData.getColumnLabel(i) / getColumnTypeName(i)</code>	Devuelve el nombre de la columna (o alias) y su tipo nativo en Oracle.

ENUNCIADO DEL RETO DE AULA

Desarrolla un visor de tablas dinámico `VisorEsquemaOracle` que liste todas las tablas del usuario `c##dam2\_user` y, para cada una de ellas, muestre el nombre de sus columnas, tipo de dato SQL y si admite valores nulos.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import java.sql.*;

public class VisorEsquemaOracle {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";

    public static void main(String[] args) {
        try (Connection conn = DriverManager.getConnection(URL, USER, PASS)) {
            DatabaseMetaData dmd = conn.getMetaData();
            System.out.println("=== VISOR DE TABLAS DEL ESQUEMA ===");

            try (ResultSet rsTablas = dmd.getTables(null, USER.toUpperCase(), "%", new String[]{"TABLE"})) {
                while (rsTablas.next()) {
                    String tabla = rsTablas.getString("TABLE_NAME");
                    System.out.println("\nTABLA: " + tabla);
                    System.out.println(" COLUMNAS:");

                    try (ResultSet rsCols = dmd.getColumns(null, USER.toUpperCase(), tabla, "%")) {
```

```
        while (rsCols.next()) {
            String colName = rsCols.getString("COLUMN_NAME");
            String colType = rsCols.getString("TYPE_NAME");
            int colSize = rsCols.getInt("COLUMN_SIZE");
            String nullable = rsCols.getString("IS_NULLABLE");
            System.out.printf("    - %-20s %-12s (%d) Nullable: %s\n",
                            colName, colType, colSize, nullable);
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}
}
```

Actividad 2.7: Configuración de Pool HikariCP y Benchmark Concurrente

AVANZADA · 35 MINUTOS

Alineación Curricular: RA2 (CE2.h) **Entorno:** Java 21 · Oracle 23c · Maven

Configuración de un pool HikariDataSource de alto rendimiento en Java 21 y comparativa de tiempos de respuesta frente al DriverManager clásico ejecutando 100 consultas consecutivas.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>new HikariConfig()</code>	Objeto de configuración de parámetros del pool: URL, credenciales, tamaños de piscina y timeouts.
<code>config.setMaximumPoolSize(10)</code>	Fija el número máximo de conexiones físicas simultáneas mantenidas contra Oracle.
<code>new HikariDataSource(config)</code>	Inicializa el pool y pre-calienta las conexiones mínimas en segundo plano.
<code>dataSource.getConnection()</code>	Toma una conexión pre-calentada del pool en microsegundos.
<code>conn.close()</code> (con HikariCP)	Interceptado por el proxy del pool: la conexión física no se destruye, sino que se devuelve a la piscina.

ENUNCIADO DEL RETO DE AULA

Añade `com.zaxxer:HikariCP:5.1.0` a tu `pom.xml`. Implementa una clase `BenchmarkPoolHikari` que ejecute 100 consultas `SELECT 1 FROM DUAL`. Primero usando `DriverManager.getConnection()` en cada iteración y luego usando `HikariDataSource.getConnection()`. Imprime el tiempo total en milisegundos de cada enfoque.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2;

import com.zaxxer.hikari.HikariConfig;
import com.zaxxer.hikari.HikariDataSource;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;

public class BenchmarkPoolHikari {
    private static final String URL = "jdbc:oracle:thin:@localhost:1521:xe";
    private static final String USER = "c##dam2_user";
    private static final String PASS = "dam2_pass";
    private static final int ITERACIONES = 50;

    public static void main(String[] args) throws Exception {
        System.out.println("=== BENCHMARK: DRIVERMANAGER vs HIKARICP (" + ITERACIONES + " consultas) ===");

        // 1. Prueba con DriverManager
```

```

long t1 = System.currentTimeMillis();
for (int i = 0; i < ITERACIONES; i++) {
    try (Connection conn = DriverManager.getConnection(URL, USER, PASS);
        PreparedStatement ps = conn.prepareStatement("SELECT 1 FROM DUAL");
        ResultSet rs = ps.executeQuery()) {
        if (rs.next()) rs.getInt(1);
    }
}
long duracionDriver = System.currentTimeMillis() - t1;
System.out.println("Tiempo con DriverManager: " + duracionDriver + " ms");

// 2. Prueba con HikariCP
HikariConfig config = new HikariConfig();
config.setJdbcUrl(URL);
config.setUsername(USER);
config.setPassword(PASS);
config.setMaximumPoolSize(10);
config.setMinimumIdle(2);

try (HikariDataSource ds = new HikariDataSource(config)) {
    long t2 = System.currentTimeMillis();
    for (int i = 0; i < ITERACIONES; i++) {
        try (Connection conn = ds.getConnection();
            PreparedStatement ps = conn.prepareStatement("SELECT 1 FROM DUAL");
            ResultSet rs = ps.executeQuery()) {
            if (rs.next()) rs.getInt(1);
        }
    }
    long duracionHikari = System.currentTimeMillis() - t2;
    System.out.println("Tiempo con HikariCP: " + duracionHikari + " ms");
    System.out.printf("Aceleración conseguida: %.1fx más rápido\n", (double) duracionDriver / duracionHikari);
}
}
}

```

Actividad 2.8: Arquitectura Limpia DAO / DTO con HikariDataSource

AVANZADA · 45 MINUTOS

Alineación Curricular: RA2 (CE2.g, CE2.h) **Entorno:** Java 21 · Oracle 23c · Maven

Diseño e implementación de un Data Access Object desacoplado: definición de ProductoDTO como Java Record, interfaz ProductoDAO, e implementación ProductoDAOJdbc inyectando javax.sql.DataSource.

INSTRUCCIONES Y MÉTODOS CLAVE

Instrucción / Método	Acción en Memoria / Servidor / Red
<code>public record ProductoDTO(...)</code>	Definición inmutable del modelo de transferencia en Java 21.
<code>public interface ProductoDAO</code>	Contrato de persistencia con métodos CRUD estándar (create, findById, findAll, update, delete).
<code>Inyección de DataSource</code>	El DAO recibe javax.sql.DataSource por constructor, desacoplándose de cómo se obtienen las conexiones.
<code>Optional</code>	Retorno expresivo que evita NullPointerException cuando no existe el registro buscado.

ENUNCIADO DEL RETO DE AULA

Implementa el patrón DAO completo para la entidad `Producto` con su correspondiente Record `ProductoDTO`, interfaz `ProductoDAO` y clase `ProductoDAOJdbc`. Prueba en una clase `AppTestDAO` la inserción, búsqueda por ID con `Optional` y listado completo.

SOLUCIÓN DE REFERENCIA Y CÓDIGO FUNCIONAL

```
package es.cifpaviles.dam2.ad.ut2.dao;

import java.math.BigDecimal;
import java.util.List;
import java.util.Optional;

// 1. DTO (Java Record)
public record ProductoDTO(Long id, String nombre, BigDecimal precio, Integer stock) {}

// 2. Interfaz DAO
interface ProductoDAO {
    void create(ProductoDTO prod);
    Optional<ProductoDTO> findById(Long id);
    List<ProductoDTO> findAll();
    void update(ProductoDTO prod);
    void delete(Long id);
}

// 3. Implementación JDBC
import javax.sql.DataSource;
import java.sql.*;
import java.util.ArrayList;
```

```

public class ProductoDAOJdbc implements ProductoDAO {
    private final DataSource dataSource;

    public ProductoDAOJdbc(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public void create(ProductoDTO prod) {
        String sql = "INSERT INTO productos (nombre, precio, stock) VALUES (?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, prod.nombre());
            ps.setBigDecimal(2, prod.precio());
            ps.setInt(3, prod.stock());
            ps.executeUpdate();
        } catch (SQLException e) {
            throw new RuntimeException("Error al crear producto: " + prod.nombre(), e);
        }
    }

    @Override
    public Optional<ProductoDTO> findById(Long id) {
        String sql = "SELECT id_producto, nombre, precio, stock FROM productos WHERE id_producto = ?";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                if (rs.next()) {
                    return Optional.of(new ProductoDTO(
                        rs.getLong("id_producto"),
                        rs.getString("nombre"),
                        rs.getBigDecimal("precio"),
                        rs.getInt("stock")
                    ));
                }
                return Optional.empty();
            }
        } catch (SQLException e) {
            throw new RuntimeException("Error al buscar producto ID " + id, e);
        }
    }

    @Override
    public List<ProductoDTO> findAll() {
        String sql = "SELECT id_producto, nombre, precio, stock FROM productos ORDER BY id_producto";
        List<ProductoDTO> lista = new ArrayList<>();
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                lista.add(new ProductoDTO(
                    rs.getLong("id_producto"),
                    rs.getString("nombre"),
                    rs.getBigDecimal("precio"),
                    rs.getInt("stock")
                ));
            }
        }
    }
}

```

```
        return lista;
    } catch (SQLException e) {
        throw new RuntimeException("Error al listar productos", e);
    }
}

@Override public void update(ProductoDTO prod) { /* DML UPDATE */ }
@Override public void delete(Long id) { /* DML DELETE */ }
}
```